Introduction to Theoretical Computer Science

CS2490 - Session 17: Closure properties of Regular Languages

Pierre Cagne

Fall 2026

September 25, 2026

Reminder

To show that a language is regular, we can:

Reminder

To show that a language is regular, we can:

1. Construct a DFA that recognizes it. (Definition)

Reminder

To show that a language is regular, we can:

1. Construct a DFA that recognizes it. (Definition)
2. Construct a NFA that recognizes it. (Subset construction)

Reminder

To show that a language is regular, we can:

1. Construct a DFA that recognizes it. (Definition)
2. Construct a NFA that recognizes it. (Subset construction)
3. Construct a regular expression that denotes it. (Two algorithms from last week)

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

- $L_1 \cup L_2$ *(union)*

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

- $L_1 \cup L_2$ *(union)*
- $L_1 \cdot L_2$ *(concatenation)*

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

- $L_1 \cup L_2$ *(union)*
- $L_1 \cdot L_2$ *(concatenation)*
- L_1^* *(Kleene star)*

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

- $L_1 \cup L_2$ *(union)*
- $L_1 \cdot L_2$ *(concatenation)*
- L_1^* *(Kleene star)*

Proof.

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs recognizing L_1 and L_2 respectively. Use the constructions seen in the algorithm that transforms a regular expression into a NFA to build an NFA recognizing $L_1 \cup L_2$, $L_1 \cdot L_2$ and L_1^*. □

Simple closure properties

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so are:

- $L_1 \cup L_2$ *(union)*
- $L_1 \cdot L_2$ *(concatenation)*
- L_1^* *(Kleene star)*

Proof.

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs recognizing L_1 and L_2 respectively. Use the constructions seen in the algorithm that transforms a regular expression into a NFA to build an NFA recognizing $L_1 \cup L_2$, $L_1 \cdot L_2$ and L_1^*. □

We say that regular languages are closed under union, concatenation, and Kleene star.

Complementation

Theorem

Let Σ be an alphabet. If L is a regular language over Σ, then so is its complement $\Sigma^* \setminus L$.

Complementation

Theorem

Let Σ be an alphabet. If L is a regular language over Σ, then so is its complement $\Sigma^* \setminus L$.

Proof.

Let $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ be a DFA recognizing L. Construct the DFA $\mathcal{A}' = (\Sigma, Q, \delta, q_0, Q \setminus F)$. It recognizes $\Sigma^* \setminus L$: a computation of $\mathcal{A}$ on a word w ends in an accepting state if and only if the computation of $\mathcal{A}'$ on w ends in a non-accepting state. □

Complementation

Theorem

Let Σ be an alphabet. If L is a regular language over Σ, then so is its complement $\Sigma^* \setminus L$.

Proof.

Let $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ be a DFA recognizing L. Construct the DFA $\mathcal{A}' = (\Sigma, Q, \delta, q_0, Q \setminus F)$. It recognizes $\Sigma^* \setminus L$: a computation of $\mathcal{A}$ on a word w ends in an accepting state if and only if the computation of $\mathcal{A}'$ on w ends in a non-accepting state. □

Question: Could we use an NFA instead of a DFA in the proof?

Intersection

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so is their intersection $L_1 \cap L_2$.

Intersection

Theorem

Let Σ be an alphabet. If L_1 and L_2 are regular languages over Σ, then so is their intersection $L_1 \cap L_2$.

Quick version.

Write $\overline{L}$ for the complement of a language L. By abstract set manipulation, we have $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Since regular languages are closed under union and complementation, they are also closed under intersection. $\square$

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.
2. Build the DFA $\mathcal{A}'_2$ recognizing $\overline{L_2}$ from $\mathcal{A}_2$.

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.
2. Build the DFA $\mathcal{A}'_2$ recognizing $\overline{L_2}$ from $\mathcal{A}_2$.
3. Use the union technique to build the NFA $\mathcal{A}'$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'_1$ and $\mathcal{A}'_2$.

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.
2. Build the DFA $\mathcal{A}'_2$ recognizing $\overline{L_2}$ from $\mathcal{A}_2$.
3. Use the union technique to build the NFA $\mathcal{A}'$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'_1$ and $\mathcal{A}'_2$.
4. Use the subset construction to build a DFA $\mathcal{A}''$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'$.

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.
2. Build the DFA $\mathcal{A}'_2$ recognizing $\overline{L_2}$ from $\mathcal{A}_2$.
3. Use the union technique to build the NFA $\mathcal{A}'$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'_1$ and $\mathcal{A}'_2$.
4. Use the subset construction to build a DFA $\mathcal{A}''$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'$.
5. Use the complement construction to build a DFA $\mathcal{A}'''$ recognizing $L_1 \cap L_2$ from $\mathcal{A}''$.

Constructive version

What if we want to build an actual automaton recognizing $L_1 \cap L_2$ from two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ recognizing L_1 and L_2, respectively?

1. Build the DFA $\mathcal{A}'_1$ recognizing $\overline{L_1}$ from $\mathcal{A}_1$.
2. Build the DFA $\mathcal{A}'_2$ recognizing $\overline{L_2}$ from $\mathcal{A}_2$.
3. Use the union technique to build the NFA $\mathcal{A}'$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'_1$ and $\mathcal{A}'_2$.
4. Use the subset construction to build a DFA $\mathcal{A}''$ recognizing $\overline{L_1} \cup \overline{L_2}$ from $\mathcal{A}'$.
5. Use the complement construction to build a DFA $\mathcal{A}'''$ recognizing $L_1 \cap L_2$ from $\mathcal{A}''$.

Question: Can we do better?

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- a state of $\mathcal{A}$ is a state of $\mathcal{A}_1$ together with a state of $\mathcal{A}_2$,

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- a state of $\mathcal{A}$ is a state of $\mathcal{A}_1$ together with a state of $\mathcal{A}_2$,
- the initial state of $\mathcal{A}$ is the initial state of $\mathcal{A}_1$ together with the initial state of $\mathcal{A}_2$,

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- a state of $\mathcal{A}$ is a state of $\mathcal{A}_1$ together with a state of $\mathcal{A}_2$,
- the initial state of $\mathcal{A}$ is the initial state of $\mathcal{A}_1$ together with the initial state of $\mathcal{A}_2$,
- a transition in $\mathcal{A}$ simulates a transition in $\mathcal{A}_1$ together with a transition in $\mathcal{A}_2$ on the same input letter,

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- a state of $\mathcal{A}$ is a state of $\mathcal{A}_1$ together with a state of $\mathcal{A}_2$,
- the initial state of $\mathcal{A}$ is the initial state of $\mathcal{A}_1$ together with the initial state of $\mathcal{A}_2$,
- a transition in $\mathcal{A}$ simulates a transition in $\mathcal{A}_1$ together with a transition in $\mathcal{A}_2$ on the same input letter,
- a state of $\mathcal{A}$ is accepting when both components are accepting.

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- $Q = Q_1 \times Q_2$ (Cartesian product)
- $q_0 = (q_0^1, q_0^2)$
- For all $(q_1, q_2) \in Q$ and $a \in \Sigma$, $\delta(a, (q_1, q_2)) = (\delta_1(a, q_1), \delta_2(a, q_2))$
- $F = F_1 \times F_2$

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- $Q = Q_1 \times Q_2$ (Cartesian product)
- $q_0 = (q_0^1, q_0^2)$
- For all $(q_1, q_2) \in Q$ and $a \in \Sigma$, $\delta(a, (q_1, q_2)) = (\delta_1(a, q_1), \delta_2(a, q_2))$
- $F = F_1 \times F_2$

By induction (on the length of w), show that: the computation of $\mathcal{A}$ on w ends in state (q_1, q_2) if and only if the computation of $\mathcal{A}_1$ on w ends in state q_1 and the computation of $\mathcal{A}_2$ on w ends in state q_2.

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- $Q = Q_1 \times Q_2$ (Cartesian product)
- $q_0 = (q_0^1, q_0^2)$
- For all $(q_1, q_2) \in Q$ and $a \in \Sigma$, $\delta(a, (q_1, q_2)) = (\delta_1(a, q_1), \delta_2(a, q_2))$
- $F = F_1 \times F_2$

By induction (on the length of w), show that: the computation of $\mathcal{A}$ on w ends in state (q_1, q_2) if and only if the computation of $\mathcal{A}_1$ on w ends in state q_1 and the computation of $\mathcal{A}_2$ on w ends in state q_2. (DO IT!)

Constructive version (cont'd)

Theorem

Let Σ be an alphabet. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two DFAs over Σ. There exists an explicit DFA $\mathcal{A}$ over Σ recognizing $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Proof.

Write $(\Sigma, Q_1, \delta_1, q_0^1, F_1)$ for $\mathcal{A}_1$ and $(\Sigma, Q_2, \delta_2, q_0^2, F_2)$ for $\mathcal{A}_2$. We build the DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$ as follows:

- $Q = Q_1 \times Q_2$ (Cartesian product)
- $q_0 = (q_0^1, q_0^2)$
- For all $(q_1, q_2) \in Q$ and $a \in \Sigma$, $\delta(a, (q_1, q_2)) = (\delta_1(a, q_1), \delta_2(a, q_2))$
- $F = F_1 \times F_2$

By induction (on the length of w), show that: the computation of $\mathcal{A}$ on w ends in state (q_1, q_2) if and only if the computation of $\mathcal{A}_1$ on w ends in state q_1 and the computation of $\mathcal{A}_2$ on w ends in state q_2. (DO IT!)
So $\mathcal{A}$ recognizes $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$.

Practice

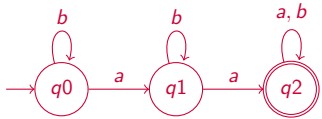
Let $\Sigma = \{a, b\}$. Construct a DFA recognizing $L_1 \cap L_2$ where:

1. Let L_1 be the language of words with at least two a's.
2. Let L_2 be the language of words with at least one b's.

Practice

Let $\Sigma = \{a, b\}$. Construct a DFA recognizing $L_1 \cap L_2$ where:

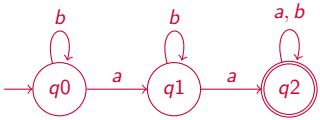
1. Let L_1 be the language of words with at least two a's.
2. Let L_2 be the language of words with at least one b's.

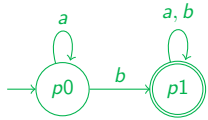


Practice

Let $\Sigma = \{a, b\}$. Construct a DFA recognizing $L_1 \cap L_2$ where:

1. Let L_1 be the language of words with at least two a's.
2. Let L_2 be the language of words with at least one b's.

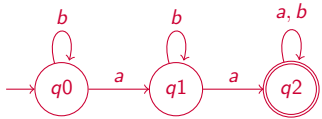


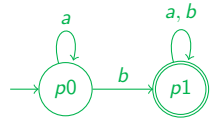


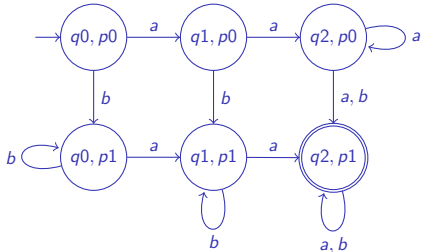
Practice

Let $\Sigma = \{a, b\}$. Construct a DFA recognizing $L_1 \cap L_2$ where:

1. Let L_1 be the language of words with at least two a's.
2. Let L_2 be the language of words with at least one b's.







Practice

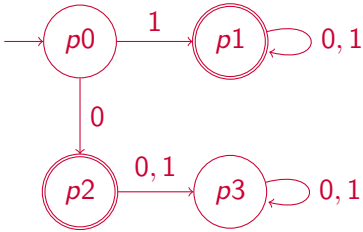
Let $\Sigma = \{0, 1\}$. Construct a DFA recognizing the binary representations (no leading zeros) of the numbers that are multiples of 3.

1. Let L_1 be the language of binary representations.
2. Let L_2 be the language of words (with potential leading zeros) representing multiples of 3.

Practice

Let $\Sigma = \{0, 1\}$. Construct a DFA recognizing the binary representations (no leading zeros) of the numbers that are multiples of 3.

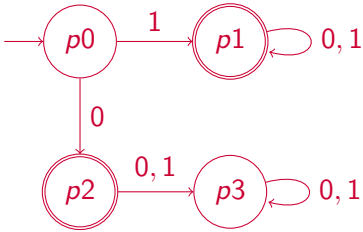
1. Let L_1 be the language of binary representations.
2. Let L_2 be the language of words (with potential leading zeros) representing multiples of 3.

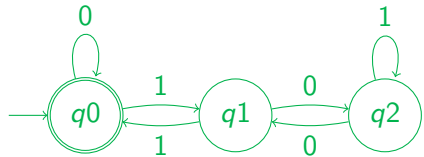


Practice

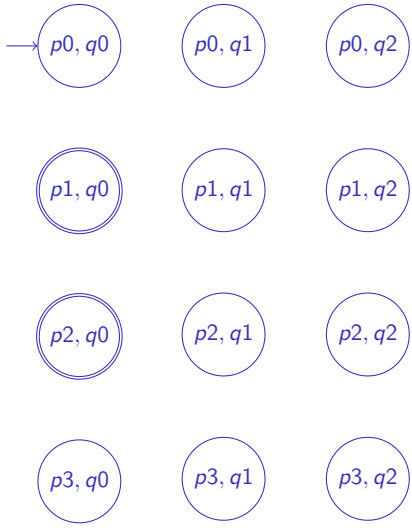
Let $\Sigma = \{0, 1\}$. Construct a DFA recognizing the binary representations (no leading zeros) of the numbers that are multiples of 3.

1. Let L_1 be the language of binary representations.
2. Let L_2 be the language of words (with potential leading zeros) representing multiples of 3.

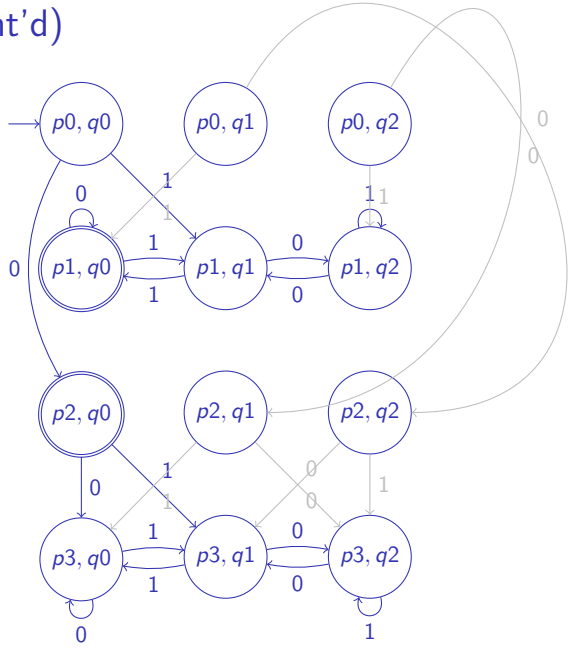




Practice (cont'd)



Practice (cont'd)



Practice (cont'd)

